Today
Welcome to your next lesson: Introduce the big O notation.

I'm preparing your personalized lesson based on what you've learned, what's coming next, and how you learn best.

Step by step, you’re moving forward.

Imagine you have a program that works with a list of numbers. That list might have 10 items… or 10 million. 🧾

Big O is a way to talk about how fast the work grows as the list gets bigger, without measuring exact seconds.

Start by watching this video: it gives a simple, visual intro to Big O 👇

(You should see it embedded in your lesson area.)

When you’re done watching (or if you already know you don’t want video), tell me:

Do you prefer we go more with visuals and stories, or more with small bits of Python code for this topic?

Use small Python examples

Nice, Python it is 🌀

Let’s start super small.

Python

```
def show_first_item(numbers):
    print(numbers[0])
```

No matter if numbers has 3 items or 3 million, this function:

does one main operation: numbers[0]
takes about the same amount of work every time

In Big O, we call this O(1) — constant time.

Rough idea: O(1) means “the work does not grow when the input grows.”

Does that idea make sense so far: this function is the same cost whether the list has 10 or 10 million items?

Yes, that makes sense

What do you think the answer is? Take your time with the question below 👇

Here’s the same function again:

```
def show_first_item(numbers):
    print(numbers[0])
```

If the list numbers grows from 10 items to 1,000,000 items, what happens to the number of steps this function does (in Big O terms)?

A.

It stays basically the same (O(1))

B.

It grows in proportion to the list size (O(n))

C.

It grows like n^2 (O(n^2))

Exactly right 🎯 It stays basically the same — O(1).

Reason: this function always does the same work:

Python

```
def show_first_item(numbers):
    print(numbers[0])  # <-- one access, one print
```

It doesn't loop over the list
It doesn't do extra work when the list is bigger

So with 10, 1,000, or 1,000,000 items, it's still just one main operation → constant time.

Next step: let's look at a tiny function that does grow with the list size (our first O(n)). Ready?

Yes, show me the O(n) example

Great, here's a simple linear example 🌀

Python

```
def print_all(numbers):
    for n in numbers:
        print(n)
```

Step by step:

The for loop goes through each item in numbers.
If there are 5 items → it prints 5 times.
If there are 1,000 items → it prints 1,000 times.

So the number of steps grows in proportion to how many items are in the list.

In Big O, this is O(n) — linear time.

Quick check (no need to overthink):

If the list has 3 items, how many times does print run in this function?

3

Exactly 💯 — 3 items → print runs 3 times.

That "do one thing per item" pattern is what O(n) means:

10 items → ~10 steps
1,000 items → ~1,000 steps
Double n → about double the work
Now I want to bring in the symbol n, because you've already been using it with your answers ☺

In this example:

Python

```
def print_all(numbers):
    for n in numbers:
        print(n)
```

Here, n in Big O means: “the size of the input” → how many items are in numbers.

So if numbers has 3 items, then n = 3; if it has 1,000 items, then n = 1000.

Does it feel clear that in Big O, n just stands for “how big the input is”?

Yes, that makes sense

Nice 👍 You’ve already nailed two big ideas:

What O(1) means (constant work)
What O(n) means (work grows with input size)
And that n = “how big the input is”

Next up is the last new shape for this lesson: $O(n^2)$ — where the work grows much faster.

Here’s a simple Python example:

Python

```
def print_pairs(numbers):
    for a in numbers:
        for b in numbers:
            print(a, b)
```

Step-by-step idea:

The outer loop goes through every item in numbers
For each a in the outer loop, the inner loop again goes through every item

So if numbers has 3 items:

Outer loop runs 3 times
Inner loop runs 3 times for each outer
Total prints = 3 × 3 = 9

That “for each item, loop over all items again” is what gives us $O(n^2)$ — quadratic time.

Let’s keep it concrete:

If numbers = [1, 2, 3, 4], so n = 4,

how many pairs (a, b) do you think get printed in print_pairs(numbers)?

You can just answer with a number.

16

Exactly right again 👏 — with 4 items, you get 16 pairs.

Reason in steps:

n = 4

Outer loop runs 4 times
Inner loop runs 4 times for each outer loop
Total prints = 4 × 4 = 16 = n²

That “n × n” growth is what O(n²) means.

So far you’ve seen:

O(1) – same work no matter how big the input
O(n) – work grows in a straight line with input size
O(n²) – work grows like a square (n times n)

Now I want to show you a simple visual comparing how these three grow as n gets bigger. Ready for a small visual to lock it in? 👀

Yes, show me the visual

Here’s a visual to compare them 👇

O(1): flat line — doesn’t care about n, always about 1 step
O(n): straight line — steps grow with n
O(n²): curved up — steps grow like n × n, much faster as n gets bigger

Try dragging the slider to larger n and watch how:

O(1) barely moves
O(n) increases steadily
O(n²) shoots up 🚀
After playing with it for a moment:

does the difference between O(1), O(n), and O(n²) feel clear, or is one of them still a bit fuzzy?

Big O: O(1) vs O(n) vs O(n²) Growth

clear

What do you think? Pick the Big O that best matches the function 👇

You’re given this Python function:

```
def mystery(numbers):
    for x in numbers:
        print(x)
```

Thinking in Big O, how does the number of steps grow as the list numbers gets bigger?

A.

It stays basically the same no matter how big the list is (O(1))

B.

It grows in direct proportion to the list size (O(n))

C.

It grows like the square of the list size (O(n²))

Nice work, Ron 🎯 You picked O(n), and that’s exactly right.

In:

Python

```
def mystery(numbers):
```

```
    for x in numbers:
        print(x)
```
The loop runs once per item in numbers.
If n items → about n prints → steps grow in direct proportion to the input size.

So the Big O is O(n): linear time.

You've now shown you understand:

What Big O is (how work grows with input size)
What n means (size of the input)
The difference between O(1), O(n), and O(n²)

One last thing before we wrap this lesson:

Want to do a quick final "which Big O is this?" question as a mini-boss fight, or are you ready to finish this lesson and move on to the next one?

Give me the boss fight

What do you think? Pick the Big O that best matches this function 👇

You have this Python function:

```
def get_first_two(numbers):
    print(numbers[0])
    print(numbers[1])
```
As the list numbers gets bigger and bigger, how does the number of steps grow in Big O terms?

A.

It stays basically the same (O(1))

B.

It grows in direct proportion to the list size (O(n))

C.

It grows like the square of the list size (O(n²))

You crushed that boss fight, Ron ⚔️😎

For:

```
def get_first_two(numbers):
    print(numbers[0])
    print(numbers[1])
```
No matter how big numbers gets, it always does 2 prints.
That's a fixed amount of work, not growing with n.
So in Big O terms, that's O(1) — constant time.

Here's what you've mastered in this lesson 🎓

Big O notation: a way to talk about how the number of steps grows as the input size n grows.
n: just means "how big the input is" (e.g., how many items are in a list).
O(1): constant time
Examples: print(numbers[0]), get_first_two(numbers)
Steps don't change when n grows.
O(n): linear time
Example: one loop over the list
Steps grow directly with n.
O(n²): quadratic time
Example: nested loop over the same list
Steps grow like n × n.
You're in a great place for future algorithm work — Big O will keep coming back, and you

already have the core shapes in your head &

If you want, we can:

Do a couple more “what Big O is this?” with code, or
Move on and build on this in the next lesson.
What feels better for you right now?